Quantum Complexity Theory

Authors: Deangelo Brasen and Sai Konkimalla and Rawley Harrison
Professor Li-Yang Tan
CS 254: Computational Complexity
Winter 2026

1 Introduction and Motivation

What happens to classical computational complexity when we change the physical model of computation?

Thus far, the complexity theory we have studied is based on the classical Turing machine, an idealized computer which can read and write symbols (deterministically) to an infinite tape according to some fixed set of rules. In fact, we are able to "augment" the power of classical Turing machines with nondeterminism, interaction, and randomness (leading to classes of decision problems like NP, IP, and BPP). This has led to a host of algorithms under these new paradigms which decide languages with no known classical polynomial-time algorithms.

When considering the physical manifestation of classical Turing machines, we observe that they generally follow the rules of classical physics, where operations are characterized as deterministic, discrete state transitions and the overall behavior of the system is fully predictable. The first shift from classical physics was in the 1920s, with *quantum mechanics*. This new theory consisted of many departures from the classical theory, including the fact that a system can exist in a *superposition* of multiple states at the same time, and it does not have a definite value until it is measured.

As the field became more established, people started to define how computation would work under this new physical theory. In the 1980s, both the quantum Turing machine [Deu85] and the idea of physical "quantum computers" were introduced [Man80] [Fey82], largely in response to the difficulty physicists had in simulating quantum systems with digital computers. Thus, the field of quantum complexity theory arose, with quantum Turing machines as the foundational mode of computation.

Given the underlying error of quantum algorithms (which we will discuss later), we can define BQP (the quantum analogue of BPP), which consists of decision problems solvable by a quantum Turing machine in polynomial time with some fixed error probability. We also make the distinction between poly-time QTMs and poly-size quantum circuits, which have similar relations as in the classical case. Further, we can define QMA (the quantum analogue of MA), where the verifier is a polynomial-time quantum computer which takes a quantum state witness, and QIP (the quantum analogue of IP), where the prover and verifier can exchange quantum messages and perform quantum operations.

In particular, more efficient quantum algorithms have been found for problems like unstructured search (with Grover's algorithm) [Gro96] and factoring (with Shor's algorithm) [Sho97], which have a quadratic and exponential speedup respectively over the best known classical algorithms. However, there are still many open questions and surprising results on how these quantum complexity classes relate to the classical complexity classes. For one, while we do know that $\mathsf{BPP} \subseteq \mathsf{BQP}$ and $\mathsf{BQP} \subseteq \mathsf{PP}$ [ADH97], we do not know the relation between BQP and PH (though we have evidence to believe that $\mathsf{BQP} \not\subset \mathsf{PH}$ using oracle separations [RT22]). One surprising result, from [Jai+09], is that $\mathsf{QIP} = \mathsf{PSPACE}$ (similar to the result that $\mathsf{IP} = \mathsf{PSPACE}$ from [Sha92]). Another very surprising result is that PostBQP, the extension of BQP which allows postselection, and PP (probabilistic polynomial time), a very large class known to contain MA, are equal [Aar04].

In this expository paper, we will introduce these results and their motivation, starting from foundational principles. In Section 2, we will introduce the necessary quantum mechanics preliminaries needed to understand quantum computing fundamentals, which we will cover in Section 3. Then, in Section 4, we will define the class BQP and some important quantum algorithms (like Grover's and Shor's) and what these algorithms say about the power of BQP compared to other complexity classes. In Section 5, we will define the classes QMA and QCMA and give some intuition on why the local Hamiltonian problem is QMA-complete. Finally, in Section 6, we will summarize the keypoints about quantum computing and quantum complexity theory covered in the paper, as well as very recent results and major open questions in the field.

2 Quantum Mechanics as a Computational Model

In this section, we will introduce the mathematical framework underlying quantum computation. Instead of presenting the full formalism of quantum mechanics, we focus only on the concepts needed to define quantum circuits and complexity classes. All of this framework is built from linear algebra and probability, which are tools that are familiar from the classical complexity theory we have covered. However, there is one major change from standard probability, since these are replaced by complex-valued amplitudes that can interfere.

2.1 Vectors and Notation

We will start with some notation. A quantum state is a vector in $\mathbb{C}^n$ (for our purposes, n will always be a power of 2). We use **Dirac notation**, which is standard in quantum computing:

- A column vector $v \in \mathbb{C}^n$ is written as $|v\rangle$ (called a **ket** vector).
- The conjugate transpose $v^\dagger$ is written as $\langle v|$ (called a **bra** vector).
- The inner product of u and v is written as $\langle u|v\rangle = u^\dagger v$.

The standard basis vector of $\mathbb{C}^2$ can be expressed as

$$|0\rangle = \begin{bmatrix}1\\0\end{bmatrix}, \ |1\rangle = \begin{bmatrix}0\\1\end{bmatrix},$$

and more generally, for a binary string $x \in \{0,1\}^n$, $|x\rangle$ represents the corresponding basis vector in $\mathbb{C}^{2^n}$. A matrix $U \in \mathbb{C}^{n\times n}$ is **unitary** if $U^\dagger U = I$, where $U^\dagger$ is the conjugate transpose of this matrix. Note that unitary matrices preserve inner products, with $\langle Uu|Uv\rangle = \langle u|v\rangle$ for all $u, v \in \mathbb{C}^n$. We say that a complex matrix $U \in \mathbb{C}^{n\times n}$ is **Hermitian** if $U = U^\dagger$. Also recall that matrices are linear operators, with $A(|x\rangle + |y\rangle) = A|x\rangle + A|y\rangle$ for all $x, y \in \mathbb{C}^n$ and $A \in \mathbb{C}^{n\times n}$.

The choice of basis vectors in our qubit representation is arbitrary, as long as the basis is orthonormal (i.e. each basis vector is unit norm and has an inner product of 0 with every other basis vector). One useful set of vectors is $|+\rangle$ and $|-\rangle$, defined as follows:

$$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \text{ and } |-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle).$$

This also turns out be another orthonormal basis (check this for yourself)!

Example 1 (Hadamard operator)**.** We define the Hadamard operator as the following matrix $H \in \mathbb{C}^{2\times 2}$:

$$H = \frac{1}{\sqrt{2}}\begin{bmatrix}1 & 1\\1 & -1\end{bmatrix}.$$

One can show that this is a unitary operator as follows (since real symmetric matrices are Hermitian, we have that $H^\dagger = H$):

$$H^\dagger H = H^2 = \frac{1}{2}\begin{bmatrix}1 & 1\\1 & -1\end{bmatrix}^2 = \frac{1}{2}\begin{bmatrix}2 & 0\\0 & 2\end{bmatrix} = \begin{bmatrix}1 & 0\\0 & 1\end{bmatrix}.$$

Evaluating H at basis vectors $|0\rangle$ and $|1\rangle$, we observe that

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = |+\rangle \text{ and } H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |-\rangle,$$

which preserves orthonormality between basis vectors. Recall that unitary matrices preserve orthonormality since they preserve inner products.

Remark 2 (Hilbert spaces)**.** $\mathbb{C}^n$ is what is called a **Hilbert space**, which simply captures the fact that we have an inner product structure on the vector space. There is an additional property of completeness, which means that any convergent sequences of vectors in this space will always converge to something in $\mathbb{C}^n$ (this is the same property that limits of convergent sequences are always well-defined in $\mathbb{R}$). This is just an additional "nice" requirement for our vector space, but we can largely ignore this property as the inner product is the most crucial for the computations in this paper.

2.2 Qubits and Multi-Qubit States

A **qubit** is the quantum analogue of a classical bit. Using the notation from before, the state of a single qubit is a unit vector in $\mathbb{C}^2$:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \text{ where } |\alpha|^2 + |\beta|^2 = 1.$$

Here, $\alpha, \beta \in \mathbb{C}$ are called **amplitudes**. When $|\alpha| = 1$ or $|\beta| = 1$, the qubit is in a definite classical state (of $|1\rangle$ or $|0\rangle$ respectively). However, when both amplitudes are nonzero, we say that the qubit is in a **superposition** of $|0\rangle$ and $|1\rangle$. For multiple qubits, we use the **tensor product** $\otimes$. If qubits A and B have states $|\psi_A\rangle = \alpha_0 |0\rangle + \alpha_1 |1\rangle$ and $|\psi_B\rangle = \beta_0 |0\rangle + \beta_1 |1\rangle$ respectively, then the joint state of the two-qubit system is

$$|\psi_A\rangle \otimes |\psi_B\rangle = \alpha_0\beta_0 |00\rangle + \alpha_0\beta_1 |01\rangle + \alpha_1\beta_0 |10\rangle + \alpha_1\beta_1 |11\rangle ,$$

which is a unit vector in $\mathbb{C}^2 \otimes \mathbb{C}^2 \cong \mathbb{C}^4$. More generally, the state of an n-qubit system is a unit vector in $(\mathbb{C}^2)^{\otimes n} \cong \mathbb{C}^{2^n}$, which can be written as

$$|\psi\rangle = \sum_{x\in\{0,1\}^n} \alpha_x |x\rangle , \text{ where } \sum_{x\in\{0,1\}^n} |\alpha_x|^2 = 1.$$

Observe the exponential growth of the state space as we add more qubits. In general, a n-qubit state requires 2^n complex amplitudes. This is why simulating quantum systems on a classical computer appears to need exponential resources, and also why quantum computers may be able to solve certain problems more efficiently than classical ones.

Note. *(Tensor product short-hand). When it is clear to do so, the tensor product $|\psi\rangle \otimes |\varphi\rangle$ is denoted by $|\psi\rangle |\varphi\rangle$ or $|\psi\varphi\rangle$ (as the tensor product between qubits is used very often).*

2.3 Three Rules of the Quantum Computational Model

We will now describe how quantum states evolve, and how we can extract information from them. This is not a full description of quantum mechanics, but instead covers the three rules relevant to quantum computation.

Rule 1: Superposition. A quantum state can be a superposition (i.e. a linear combination) of classical basis states. Unlike the probabilistic case, where the system is "really" in one state and we just do not know which, superpositions have physical consequences which can be observed through interference effects.

Rule 2: Unitary Evolution. The state of a quantum system evolves with multiplication by a unitary matrix. In other words, if the current state is $|\psi\rangle$, and we apply unitary matrix U, the state then becomes $U|\psi\rangle$.

The fact that U must be a unitary matrix has an important consequence: every quantum operation is **reversible**, since $U^{-1} = U^\dagger$ always exists. This means, for instance, that there is no direct quantum analogue to the classical AND gate, since two input bits are mapped to one output bit (resulting in a non-injective operation). As we will see in Section 3, the constraint from reversibility does not completely limit us, since it is possible to make classical computation reversible with only polynomial-time overhead.

Rule 3: Measurement. To extract classical information from a quantum state, we can **measure** it. When we measure a n-qubit state $|\psi\rangle = \sum_x \alpha_x |x\rangle$ under the basis $\{|x\rangle : x \in \{0,1\}^n\}$, we observe outcome x with probability $|\alpha_x|^2$. After measurement, the state **collapses** to $|x\rangle$ and the remaining amplitudes are lost.

Measurement is where probability enters quantum computation (which is similar to random coin flips in BPP algorithms). However, measurement is irreversible and destroys information, which means that once we measure, we cannot fully recover the superposition. Thus, when designing quantum algorithms, we need to be careful that the amplitudes of correct answers is large enough before measurement occurs.

Note. *(Measurement is unique under global phase). Consider two quantum states $|\psi\rangle$ and $|\varphi\rangle$. Note that the measurement probability is unique up to global phase. That is, if*

$$|\psi\rangle = \sum_{x\in\{0,1\}^n} \alpha_x |x\rangle \text{ and } |\varphi\rangle = \sum_{x\in\{0,1\}^n} \beta_x |x\rangle ,$$

then

$$|\alpha_x|^2 = |\beta_x|^2 \ \forall x \in \{0,1\}^n \iff \exists\ \theta \in [0, 2\pi) \text{ such that } |\psi\rangle = e^{i\theta} |\varphi\rangle .$$

We denote this similarity in measurement probability by $|\psi\rangle \sim |\varphi\rangle$.

2.4 Interference

The property of quantum computation which enables speedups is known as **interference**, where amplitudes have the ability to cancel or build on each other. Since amplitudes are complex numbers (and can thus be negative), we get **constructive interference** when amplitudes of the same sign add together, and **destructive interference** when amplitudes of different signs add together and cancel out. To demonstrate this explicitly, consider the following example:

Example 3 (Interference with Hadarmard operator)**.** Recall the Hadamard operator H (which we showed was a single-qubit unitary matrix) defined as

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}.$$

If we apply H to $|0\rangle$, we get the uniform superposition given by $H|0\rangle = |+\rangle$. If we apply H another time, we get

$$H|+\rangle = H\left(\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)\right) = \frac{1}{2}\Big((|0\rangle + |1\rangle) + (|0\rangle - |1\rangle)\Big) = |0\rangle.$$

Here, we see that the amplitudes for $|1\rangle$ cancel out, while the amplitudes for $|0\rangle$ reinforce. This is a key principle used in every quantum algorithm, where the computation is designed so that the amplitudes of wrong answers cancel out, while the amplitudes of correct answers accumulate.

Example 4 (Deutsch's problem)**.** Suppose we are given a function $f : \{0,1\} \to \{0,1\}$ as a black box, and we wish to determine whether f is *constant* ($f(0) = f(1)$) or *balanced* ($f(0) \neq f(1)$). Classically, we must evaluate f twice (on both inputs) to answer this question. Surprisingly, a quantum computer can answer it with a single query to f.

The algorithm works as follows. We use two qubits, initialized to $|0\rangle|1\rangle$. First, we apply H to both qubits, producing $|+\rangle|-\rangle$. Next, we apply the oracle U_f, which maps $|x\rangle|y\rangle \mapsto |x\rangle|y \oplus f(x)\rangle$ (where $\oplus$ is the XOR operation on the states). Note that U_f is unitary (try to show this yourself!), so this is a valid operation on the qubits. Since the second qubit input is $|-\rangle$, we get (with some casework on $f(x)$) that

$$U_f|x\rangle|-\rangle = (-1)^{f(x)}|x\rangle|-\rangle.$$

This is known as **phase kickback**, where information about $f(x)$ is moved out of the target register and into a global phase on the input register $|x\rangle$. Since $|x\rangle = |+\rangle$, we get that

$$U_f|+\rangle|-\rangle = \left(\frac{(-1)^{f(0)}}{\sqrt{2}}|0\rangle + \frac{(-1)^{f(1)}}{\sqrt{2}}|1\rangle\right) \otimes |-\rangle.$$

Finally, we apply H to the first qubit and measure. If f is constant, the amplitudes are equal (both $+$ or both $-$), and after H the state is $\pm|0\rangle$, so we measure 0 with certainty. If f is balanced, the amplitudes are opposite, and after H the state is $\pm|1\rangle$, so we measure 1 with certainty. In both cases, a single query suffices.

The key mechanism here is interference: the Hadamard gate after the oracle combines the two amplitudes $(-1)^{f(0)}$ and $(-1)^{f(1)}$, and depending on whether they agree or disagree, one output is reinforced while the other is canceled. This is the same constructive/destructive interference pattern we saw with H applied twice, now used to compute a property of a function.

Remark 5 (Revisiting BPP)**.** Recall that there are two equivalent ways to think about randomized computation in BPP. The first is operational: a deterministic Turing machine is augmented with an additional tape of random bits, and the machine's behavior depends on the random string it receives. The second is more abstract but will be useful for comparison with quantum computation: rather than tracking a single configuration that evolves stochastically, we maintain a *probability*

distribution over all possible configurations. That is, the state of the computation at any time step is a vector $(\alpha_x)_{x\in\{0,1\}^n}$, where n is the total number of bits of state used by the machine, $\alpha_x \geq 0$ is the probability of being in configuration x, and $\sum_{x\in\{0,1\}^n} \alpha_x = 1$. Each step of the computation applies a stochastic matrix to this vector, and the final output is obtained by sampling from the resulting distribution.

This second interpretation reveals a striking parallel with quantum computation. A quantum state on n qubits is also a vector $(\alpha_x)_{x\in\{0,1\}^n}$ with $\sum_{x\in\{0,1\}^n} |\alpha_x|^2 = 1$, each step applies a matrix (now unitary rather than stochastic), and the output is obtained by sampling (now with probabilities $|\alpha_x|^2$ rather than α_x). The essential difference is that the entries of a probability distribution are non-negative reals, while quantum amplitudes are complex numbers that can be negative or differ in phase.

This means that in a randomized computation, when two computational paths lead to the same configuration, their probabilities always add, so there is no possibility of cancellation. In quantum computation, the amplitudes of two paths may have opposite signs and cancel each other out (destructive interference) or reinforce each other (constructive interference). This capacity for interference is the fundamental source of potential quantum speedup, and every quantum algorithm we will encounter exploits it.

2.5 Entanglement

Not every multi-qubit state can be written as a tensor product of individual qubit states. A state that cannot be decomposed in this way is called **entangled**. The simplest example is the **Bell state**:

$$\left|\Phi^+\right\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle).$$

To see that this is entangled, assume for sake of contradiction that $|\Phi^+\rangle = (\alpha|0\rangle + \beta|1\rangle) \otimes (\gamma|0\rangle + \delta|1\rangle)$. Expanding, we would need $\alpha\gamma = \frac{1}{\sqrt{2}}$, $\alpha\delta = 0$, $\beta\gamma = 0$, and $\beta\delta = \frac{1}{\sqrt{2}}$. From the second equation, $\alpha = 0$ or $\delta = 0$, but either choice contradicts the first or fourth equation.

The Bell state has a striking property: measuring the first qubit gives 0 or 1 each with probability $\frac{1}{2}$, and the second qubit is then guaranteed to be in the same state. That is, the two qubits are perfectly correlated, even though each qubit individually appears uniformly random. This kind of non-classical correlation is a key resource in quantum computation and quantum communication protocols.

For the purposes of this paper, the most important consequence of entanglement is that a general n-qubit state requires 2^n parameters to describe, while an unentangled (product) state requires only $2n$ (since each qubit has 2 complex numbers). This exponential gap is what makes classical simulation of quantum systems generically hard, and it is the source of the potential computational advantage of quantum computing.

2.6 The No-Cloning Theorem

We conclude this section with a result that highlights a fundamental difference between classical and quantum information.

Theorem 6 (No-cloning theorem [WZ82]). There is no unitary operation U that maps $|\psi\rangle|0\rangle \to |\psi\rangle|\psi\rangle$ for all quantum states $|\psi\rangle$.

Proof. Suppose such a U exists. Then for any two states $|\psi\rangle$ and $|\varphi\rangle$:

$$U|\psi\rangle|0\rangle = |\psi\rangle|\psi\rangle \text{ and } U|\varphi\rangle|0\rangle = |\varphi\rangle|\varphi\rangle.$$

Taking the inner product of both sides:

$$\langle\psi|\varphi\rangle \cdot \langle 0|0\rangle = \langle\psi|\varphi\rangle^2.$$

Since $\langle 0|0\rangle = 1$, we have $\langle\psi|\varphi\rangle = \langle\psi|\varphi\rangle^2$, which implies $\langle\psi|\varphi\rangle \in \{0, 1\}$. That is, U can only clone states that are either identical or orthogonal; it cannot clone arbitrary unknown states. □

This result has significant consequences for quantum complexity theory. In classical computation, a witness string (as in NP verification) can be freely copied for analysis. However, in the quantum setting, a quantum witness cannot be copied, which means that techniques relying on independent copies (e.g. majority voting for boosting) require more careful treatment. We will encounter this issue when we discuss the class QMA in Section 5.

3 Quantum Circuits and Classical Simulation

With the mathematical framework of Section 2, we cab now define the quantum circuit model: the standard computational model for quantum complexity theory. The structure of this section is similar to how we developed classical circuits: we will first define gates, build circuits from them, establish universality, and then connect the circuit model to Turing machines. We will also show that classical computation embeds cleanly into quantum computation, which will later imply $\mathsf{BPP} \subseteq \mathsf{BQP}$.

3.1 Quantum Gates

Definition 7. A **quantum gate** on k qubits is a $2^k \times 2^k$ unitary matrix. Gates are the quantum analogues of classical Boolean gates (AND, OR, NOT), with the key difference that they must be unitary and therefore reversible.

Single-qubit gates. The most important single-qubit gates are the following:

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad T = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix}.$$

The X gate is the quantum analogue of classical NOT: it swaps $|0\rangle$ and $|1\rangle$. The Z gate applies a phase flip, mapping $|1\rangle \mapsto -|1\rangle$ while leaving $|0\rangle$ unchanged. We have already encountered the Hadamard gate H in Section 2, where we saw that it creates superpositions and enables interference. The T gate adds a phase of $e^{i\pi/4}$ to the $|1\rangle$ component (it will become more useful when discussing universality).

Note that X, Z, and H are their own inverses (they are both unitary and Hermitian), while $T^\dagger$ (the inverse of T) applies a phase of $e^{-i\pi/4}$.

Multi-qubit gates. The most important multi-qubit gate is the **controlled-NOT** (CNOT) gate, a two-qubit gate defined by

$$\text{CNOT} : |a\rangle |b\rangle \mapsto |a\rangle |a \oplus b\rangle ,$$

where $\oplus$ denotes addition modulo 2 (i.e. the XOR operation). We can represent the CNOT gate in matrix form (with respect to the basis $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$) to get

$$\text{CNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

The first qubit is the control: if it is $|0\rangle$, nothing happens to the second qubit; if it is $|1\rangle$, the second qubit is flipped. One can verify that CNOT is unitary. The CNOT gate is significant because it is the simplest gate that can create entanglement (verify that applying CNOT to $|+\rangle |0\rangle$ produces the Bell state $|\Phi^+\rangle$ from Section 2.5)!

More generally, a **controlled-U** gate applies a unitary U to a target register conditioned on a control qubit being $|1\rangle$. We will use controlled operations quite a lot in the quantum algorithms of Section 4.

3.2 Quantum Circuits

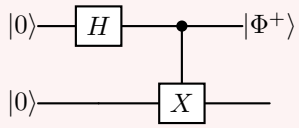
We will now assemble gates into circuits.

Definition 8. A **quantum circuit** on n qubits over a gate set $\mathcal{G}$ is a sequence of gates $G_1, G_2, \ldots, G_m$, where each $G_i \in \mathcal{G}$ acts on a specified subset of the n qubits (and acts as the identity on the remaining qubits). The circuit computes the unitary transformation $U = G_m G_{m-1} \cdots G_1$. The **size** of the circuit is m (the number of gates), and the **depth** is the number of time steps when gates on disjoint qubits are allowed to act in parallel.

A quantum computation proceeds as follows: initialize n qubits to $|0\rangle^{\otimes n}$, apply the circuit U, and measure some subset of the qubits in the computational basis. The output is the measurement outcome, which is probabilistic (determined by the amplitudes of the final state). We thus see how classical circuits are analogous to quantum circuits. A classical Boolean circuit is a DAG of AND, OR, and NOT gates that maps n input bits to m output bits. A quantum circuit is a sequence of unitary gates that maps n input qubits to n output qubits (the number of qubits is preserved, since unitary maps are square matrices).

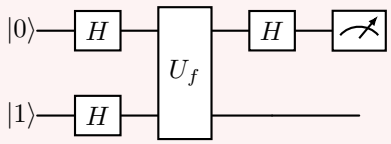
Just as we consider families of classical circuits $\{C_n\}_{n\in\mathbb{N}}$ (one circuit for each input length), we consider **families of quantum circuits** $\{Q_n\}_{n\in\mathbb{N}}$. A circuit family is **uniform** if there exists a polynomial-time classical Turing machine that, on input 1^n, outputs a description of Q_n. Uniformity ensures that the circuits themselves can be efficiently constructed, preventing us from "hiding" computational power in the circuit descriptions. This is the same role uniformity plays in classical circuit complexity, and it will be important when we define BQP.

Remark 9 (Visual representation of quantum circuits)**.** As we start representing more complex quantum algorithms, it becomes useful to have a visualization of the algorithm. Thus, similar to diagrams of logical gates in classical circuits, we can represent quantum circuits with input qubits, various quantum gates, and measurements. As an example, consider the following representation of the CNOT gate evaluated on $|+\rangle|0\rangle$:



The input qubits are $|0\rangle|0\rangle$, with the first qubit passed through a Hadamard gate and then to the CNOT gate and the second qubit passed directly to the CNOT gate. Recall that $H|0\rangle = |+\rangle$, and as discussed before, $\text{CNOT}(|+\rangle|0\rangle) = |\Phi^+\rangle$.

We can also create a compact representation of Deutsch's algorithm from Example 4!



Here, the symbol  represents a measurement after applying the previous quantum gates to the initial quantum state. Also, similar to CNOT, the oracle U_f is a quantum gate which takes two qubits as inputs and similarly gives two qubits as output, though in this case, we only cared about the first qubit for the output.

3.3 Universality and Approximation

In classical computation, the gate set $\{\text{AND}, \text{OR}, \text{NOT}\}$ is **universal**: any Boolean function can be computed exactly by a circuit over these gates. The situation in quantum computing is analogous but a bit more subtle, because we are now working over a continuous space of unitary matrices rather than a finite set of Boolean functions.

Definition 10. A gate set $\mathcal{G}$ is **(approximately) universal** for quantum computation if, for any n-qubit unitary U and any $\epsilon > 0$, there exists a circuit C over $\mathcal{G}$ such that $\|C - U\| \leq \epsilon$, where $\|\cdot\|$ denotes the operator norm.

The standard universal gate set for quantum computation is $\{H, \text{CNOT}, T\}$.

Theorem 11 (Universal quantum gate set [Boy+99]). The gate set $\{H, \text{CNOT}, T\}$ is approximately universal for quantum computation. That is, any n-qubit unitary can be approximated to arbitrary precision by a circuit composed of H, CNOT, and T gates.

The shift from exact to approximate universality raises an immediate concern: does the overhead of approximation destroy polynomial-time efficiency? The answer is no, thanks to the following result.

Theorem 12 (Solovay-Kitaev theorem [DN05]). Let $\mathcal{G}$ be any finite universal gate set. Then any single-qubit unitary can be approximated to within ϵ (in operator norm) by a circuit over $\mathcal{G}$ of size $O(\log^c(1/\epsilon))$, where c is a constant (approximately 3.97, which has been improved in later work).

The Solovay-Kitaev theorem has two important consequences. First, the cost of approximation is **polylogarithmic** in the desired precision. This means that achieving exponentially good approximation requires only a polynomial blowup in circuit size. In particular, if a quantum algorithm uses T gates and requires precision $\epsilon = \exp(-\operatorname{poly}(n))$ (which is more than sufficient for BQP computations), the overhead is polynomial. Second, the choice of universal gate set does not matter up to polynomial factors: any two finite universal gate sets can simulate each other with at most polylogarithmic overhead per gate. This is the quantum analogue of the classical fact that $\{\text{AND}, \text{OR}, \text{NOT}\}$ and $\{\text{NAND}\}$ differ by at most constant factors.

3.4 Embedding Classical Computation

We will now show that quantum circuits can efficiently simulate classical circuits. From this, we can conclude that classical computation is just a special case of quantum computation.

The main obstacle is reversibility. Classical gates like AND are not reversible (as discussed in Section 2.3), and therefore have no direct unitary analogue. The solution comes in two steps: first, we show that classical computation can be made reversible; then we observe that reversible classical gates are automatically unitary.

Step 1: Reversible classical computation. The key gate is the **Toffoli gate**, a three-bit gate defined by

$$\text{Toffoli} : (a, b, c) \mapsto (a, b, c \oplus (a \wedge b)).$$

The Toffoli gate flips the third bit if and only if the first two bits are both 1, and it is its own inverse (applying it twice restores the original state). Note that the Toffoli gate is **classically universal for reversible computation**. This means that, by setting appropriate input bits to constants, it can simulate AND (set $c = 0$, so the output on the third bit is $a \wedge b$), NOT (set $a = b = 1$, so the output is $\bar{c}$), and FANOUT/COPY (set $b = 1$ and $c = 0$, yielding $(a, 1, a)$).

A classical result from Bennett [Ben73] shows that any classical circuit of size s can be simulated by a reversible circuit of size $O(s)$ using $O(s)$ additional **ancilla bits** (initialized to 0 and returned to 0 at the end of the computation). The idea is to first compute forward (accumulating garbage in the ancillas), copy the output, and then "uncompute" by running the forward computation in reverse to clean up the ancillas. This uncomputation step is important for quantum computation, where leftover garbage bits would become entangled with the output and corrupt the computation.

Step 2: Reversible gates are unitary. Since the Toffoli gate is a permutation on $\{0, 1\}^3$, its matrix representation (in the computational basis) is a permutation matrix, which is automatically unitary. Therefore, the Toffoli gate is a valid three-qubit quantum gate. More generally, any reversible classical gate corresponds to a permutation matrix and hence a valid quantum gate.

Combining the two steps: given any classical circuit C of size s, we can convert it to a reversible circuit of size $O(s)$ using Toffoli gates, and then interpret each Toffoli gate as a quantum gate. The resulting quantum circuit simulates C exactly on computational basis states. More formally, we have the following.

Proposition 13. For any classical (possibly randomized) circuit of size s, there exists a quantum circuit of size $O(s)$ that computes the same function on classical inputs. In particular, any language in BPP can be decided by a polynomial-size quantum circuit family with the same error bound. That is, $\mathsf{BPP} \subseteq \mathsf{BQP}$.

The "randomized" case follows by noting that random coin flips can be simulated with quantum gates: applying H to $|0\rangle$ produces $|+\rangle$ which gives $|0\rangle$ or $|1\rangle$ with uniform probability.

3.5 Quantum Turing Machines and the Circuit-TM Equivalence

Just as in classical complexity theory, we have two equivalent computational models: circuits and Turing machines. The **quantum Turing machine** (QTM), introduced by Deutsch [Deu85] and formalized by Bernstein and Vazirani [BV93], is the direct quantum analogue of a probabilistic Turing machine. Instead of stochastic transitions, a QTM applies unitary transitions to a state that is a superposition of classical configurations (head position, tape contents, internal state).

We will not give the full formal definition of a QTM here, since quantum circuits are much easier to work with, and will form the basis for the quantum algorithms we will discuss alter. However, the following equivalence is important:

Theorem 14 (Quantum circuit-TM equivalence [Chi93; BV93])**.** A language L is decidable by a polynomial-time quantum Turing machine with bounded error if and only if L is decidable by a polynomial-size uniform quantum circuit family with bounded error.

The proof follows the same strategy as the classical equivalence between Turing machines and uniform circuit families. The one subtlety specific to the quantum case is approximation. A QTM's transition function involves amplitudes that may be irrational, and the circuit-to-TM direction must handle gate approximation. The Solovay-Kitaev theorem (Theorem 11) ensures that this approximation overhead is at most polylogarithmic, preserving polynomial-time bounds.

This equivalence justifies defining complexity classes in terms of either model. We will primarily use the circuit model in the remainder of this paper, as it is more convenient for describing algorithms and defining classes like BQP.

3.6 Summary

We can now observe, from a more general view, the parallels between classical and quantum circuits:

Concept	**Classical**	**Quantum**
Basic unit	Bit $\in \{0,1\}$	Qubit $\in \mathbb{C}^2$, $\|\cdot\| = 1$
State on n units	String $x \in \{0,1\}^n$	Unit vector $\|\psi\rangle \in \mathbb{C}^{2^n}$
Operation	Boolean function	Unitary matrix
Randomness	Coin flip (prob. distribution)	Measurement (amplitude squared)
Universal gates	$\{\text{AND}, \text{OR}, \text{NOT}\}$ (exact)	$\{H, \text{CNOT}, T\}$ (approximate)
Reversibility	Optional (Toffoli)	Required (unitary property)
Copying	Free (FANOUT)	Impossible (no-cloning)
TM $\leftrightarrow$ circuits	Poly-time TM $\equiv$ uniform circuits	Poly-time QTM $\equiv$ uniform q-circuits

The key takeaway is that quantum computation is not an entirely foreign paradigm. It is classical computation with two modifications: the state space is enlarged from $\{0,1\}^n$ to the unit sphere in $\mathbb{C}^{2^n}$, and operations are restricted to be unitary (and hence reversible). Everything else carries over with only minor modifications. Now, given the much larger state space and the ability of interference, can we actually get more computational power?

4 BQP and Quantum Algorithms

Now, given the more formal definition of quantum circuits from Section 3, we can now define what it means for a problem to be "efficiently computable" with a uniform polynomial-sized quantum circuit. It's now time to define our first quantum complexity class, BQP!

Definition 15. A language L is in BQP (or **bounded-error quantum polynomial time**) if and only if there exists a poly-time uniform family of quantum circuits $\{Q_n\}_{n\in\mathbb{N}}$ such that

- For all $n \in \mathbb{N}$, Q_n takes n input qubits (initialized to $|x\rangle$ for an input $x \in \{0,1\}^n$) along with $\text{poly}(n)$ ancilla qubits (initialized to $|0\rangle$), and a designated output qubit is measured to produce the decision,
- For all $x \in L$, $\Pr\big[Q_{|x|}(x) = 1\big] \geq \frac{2}{3}$,
- For all $x \notin L$, $\Pr\big[Q_{|x|}(x) = 0\big] \geq \frac{2}{3}$.

Remark 16. Recall that the BQP definition is equivalent to the one using poly-time QTMs (by Theorem 14).

Note the similarities between the definitions of BQP and BPP. Just as in the BPP case, the success probability can be boosted: given a circuit that succeeds with probability at least $\frac{1}{2} + \frac{1}{\text{poly}(n)}$, we can run $\text{poly}(n)$ independent copies of the circuit and take a majority vote over the measurement outcomes. By the Chernoff bound, this drives the error probability down to $\exp(-\text{poly}(n))$.

Note that there are no issues from the no-cloning theorem (Theorem 6) since we are running independent executions of the quantum circuit rather than copying the initial quantum state (that is, the boosting is done over the classical measurement outcomes). As a result, the constant $\frac{2}{3}$ in the definition is arbitrary, where any constant strictly greater than $\frac{1}{2}$ (or even $\frac{1}{2} + \frac{1}{\text{poly}(n)}$) yields the same class of decision problems.

In the rest of this section, we will look at interesting quantum algorithms (specifically ones for unstructured search and factoring), a tight lower bound on unstructured search (the BBBV result), and their consequences on how BQP relates to various classical complexity classes like PP and PSPACE.

4.1 Unstructured Search and Grover's Algorithm

The first problem we will discuss is the unstructured search problem, defined as follows:

Definition 17. Consider a search space of size $N = 2^n$, indexed by bit strings $x \in \{0,1\}^n$. Let $f : \{0,1\}^n \to \{0,1\}$ be a function accessed only as a **black-box oracle**, where $f(x) = 1$ if x is a solutions, and $f(x) = 0$ otherwise. Let $M = |\{x \in \{0,1\}^n : f(x) = 1\}|$ (i.e. M is the total number of solutions in the search space). Then, the **unstructured search problem** is:

- (Decision version): Given oracle access to f, decide whether $M \geq 1$ (i.e. does there exist $x \in \{0,1\}^n$ such that $f(x) = 1$)?
- (Search version): Given oracle access to f and the promise that $M \geq 1$, find an $x \in \{0,1\}^n$ such that $f(x) = 1$).

Remark 18. This search problem is called *unstructured* since no information about the location of the solutions is available beyond the direct oracle access to f. That is, knowing the value of $f(x)$ for some $x \in \{0,1\}^n$ gives no information about $f(x')$ for any other x'.

Classical complexity. How efficiently can classical algorithms solve unstructured search? In the worst case, a deterministic algorithm must query f on every input before it can be certain a solution exists, which requires $\Omega(N)$ queries. A randomized algorithm can do better in expectation: by querying f on uniformly random inputs, we find a solution after $O(N/M)$ queries in expectation (and $\Omega(N/M)$ queries

are necessary, since each random query hits a solution with probability M/N). In the worst case of a single solution ($M = 1$), the randomized complexity is therefore $\Theta(N)$.

Quantum speedup. Grover's algorithm [Gro96] achieves a quadratic improvement: it finds a solution using only $O(\sqrt{N/M})$ oracle queries. We will prove this for the case $M = 1$ (a unique solution), where the algorithm is cleanest. The general case uses the same ideas with minor modifications (see [NC10] for the full proof).

Theorem 19 (Grover's algorithm [Gro96])**.** There exists a quantum algorithm that, given oracle access to $f : \{0,1\}^n \to \{0,1\}$ with the promise that there is a unique $x^* \in \{0,1\}^n$ satisfying $f(x^*) = 1$, finds x^* with high probability using $O(\sqrt{N})$ oracle queries, where $N = 2^n$.

Proof. We describe the algorithm and analyze its correctness.

Setup. The algorithm operates on n qubits and uses two special operators. The first is the **phase oracle** $\mathcal{O}_f$, which marks the solution by flipping its phase:

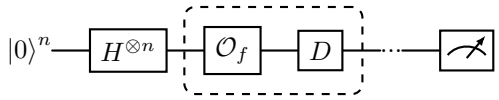
$$\mathcal{O}_f |x\rangle = (-1)^{f(x)} |x\rangle = \begin{cases} -|x\rangle & \text{if } f(x) = 1, \\ |x\rangle & \text{if } f(x) = 0. \end{cases}$$

Note that $\mathcal{O}_f$ can be implemented using a single query to the standard oracle $U_f : |x\rangle |b\rangle \mapsto |x\rangle |b \oplus f(x)\rangle$ together with the phase kickback trick from Example 4 (apply U_f with the target qubit in the state $|-\rangle$). The second operator is the **diffusion operator**:

$$D = H^{\otimes n}(2 |0^n\rangle \langle 0^n| - I)H^{\otimes n} = 2 |+^n\rangle \langle +^n| - I,$$

where $|+^n\rangle = H^{\otimes n} |0^n\rangle = \frac{1}{\sqrt{N}} \sum_{x \in \{0,1\}^n} |x\rangle$ is the uniform superposition. This operator reflects the state about $|+^n\rangle$.

The Grover iterate. The algorithm applies the **Grover iterate** $G = D \cdot \mathcal{O}_f$ repeatedly, starting from the state $|+^n\rangle$ (which can be initialized by applying $H^{\otimes n}$ to the state $|0^n\rangle$). Visually, the quantum circuit looks like the following, with some number of iterations (to be determined) of the Grover iterate:



Geometric analysis. The key insight is that the entire computation takes place in a two-dimensional subspace. Define

$$|\psi_0\rangle = \frac{1}{\sqrt{N-1}} \sum_{x \neq x^*} |x\rangle \text{ and } |\psi_1\rangle = |x^*\rangle .$$

The initial state can be written as

$$|+^n\rangle = \sqrt{\frac{N-1}{N}} |\psi_0\rangle + \frac{1}{\sqrt{N}} |\psi_1\rangle = \cos\theta |\psi_0\rangle + \sin\theta |\psi_1\rangle ,$$

where θ is defined by $\sin\theta = \frac{1}{\sqrt{N}}$. Since $|\psi_0\rangle$ and $|\psi_1\rangle$ are orthonormal, we can visualize the state as a unit vector in the plane spanned by $\{|\psi_0\rangle , |\psi_1\rangle\}$ at angle θ from the $|\psi_0\rangle$ axis.

Now we can examine what the Grover iterate does geometrically. The phase oracle $\mathcal{O}_f$ negates the $|\psi_1\rangle$ component while preserving the $|\psi_0\rangle$ component, which is a reflection through the $|\psi_0\rangle$ axis in the $\{|\psi_0\rangle , |\psi_1\rangle\}$ plane. The diffusion operator $D = 2 |+^n\rangle \langle +^n| - I$ is a reflection through $|+^n\rangle$ (the initial state). The composition of two reflections is a rotation: specifically, $G = D \cdot \mathcal{O}_f$ rotates the state by angle 2θ toward $|\psi_1\rangle$ in the $\{|\psi_0\rangle , |\psi_1\rangle\}$ plane. By induction, after k applications of G:

$$G^k |+^n\rangle = \cos((2k+1)\theta) |\psi_0\rangle + \sin((2k+1)\theta) |\psi_1\rangle .$$

Choosing the number of iterations. We want the final state to be close to $|\psi_1\rangle = |x^*\rangle$, which requires $\sin((2k+1)\theta) \approx 1$, i.e., $(2k+1)\theta \approx \frac{\pi}{2}$. Since $\sin\theta = \frac{1}{\sqrt{N}}$ and $\theta \approx \frac{1}{\sqrt{N}}$ for large N (using the small-angle approximation), we need

$$k \approx \frac{\pi}{4}\sqrt{N} - \frac{1}{2},$$

so we set $k = \lfloor \frac{\pi}{4}\sqrt{N} \rfloor$. Each iteration uses one query to $\mathcal{O}_f$ (and hence one query to f), so the total query complexity is $O(\sqrt{N})$. Measuring the final state in the computational basis yields x^* with probability $\sin^2((2k+1)\theta) \geq 1 - O(1/N)$, which is sufficient. □

Optimality of Grover's algorithm. A natural question is whether the $O(\sqrt{N})$ query complexity can be improved. The following result shows that it cannot:

Theorem 20 (BBBV lower bound [Ben+97])**.** Any quantum algorithm that solves the unstructured search problem (with $M = 1$ solution among N elements) with constant success probability must make $\Omega(\sqrt{N})$ queries to the oracle.

Proof sketch. The key idea is that each oracle query can only change the quantum state by a limited amount. Formally, consider running the algorithm on a uniformly random oracle f that has a single solution x^* chosen uniformly at random, versus running it on the identically-zero oracle $f \equiv 0$. After k queries, the states of the algorithm under these two oracles have inner product at least $1 - O(k^2/N)$; this follows because each query can "detect" the location of x^* with amplitude at most $O(1/\sqrt{N})$ (since the algorithm's state has at most $O(1/\sqrt{N})$ amplitude on $|x^*\rangle$ on average over the random choice of x^*), and these perturbations accumulate over k queries. For the two final states to be distinguishable with constant probability, we need $k^2/N = \Omega(1)$ (i.e. $k = \Omega(\sqrt{N})$). □

Remark 21 (Implications for NP)**.** The BBBV lower bound has important consequences for the power of quantum computation relative to NP. Consider a Boolean formula φ on n variables. We can define the function $f : \{0,1\}^n \to \{0,1\}$ by $f(x) = 1$ if and only if x satisfies φ. Unlike in the abstract oracle setting, here f is not a black box. We have the explicit circuit for φ, which we can simulate with quantum gates using the classical embedding from Section 3.4. Grover's algorithm can then search over all $N = 2^n$ assignments in $O(\sqrt{N}) = O(2^{n/2})$ time, providing a quadratic speedup over the $O(2^n)$ brute-force classical approach.

However, this is still exponential time, and the BBBV bound tells us we cannot do better using this unstructured approach. More precisely, the BBBV bound is an *oracle* lower bound: it shows that any quantum algorithm treating f as a black box requires $\Omega(\sqrt{N})$ queries. While this does not rule out quantum algorithms that exploit the structure of specific NP-complete problems (just as classical algorithms can exploit structure to beat brute force for some instances), it provides strong evidence that quantum computers cannot solve NP-complete problems in polynomial time. In particular, if one could solve NP-complete problems in quantum polynomial time, this would imply $\mathsf{NP} \subseteq \mathsf{BQP}$, which is widely believed to be false. Grover's quadratic speedup appears to be the best that quantum computation offers for generic search problems.

4.2 Shor's Algorithm and the Nontriviality of BQP

The next problem we will discuss is the integer factoring problem.

Definition 22. The **factoring problem** is: given an integer $N > 1$, find a nontrivial factor of N (i.e., an integer $1 < p < N$ such that $p \mid N$), or determine that N is prime.

Classical complexity. The best known classical algorithms for factoring are far from polynomial time. Trial division takes $O(\sqrt{N}) = O(2^{n/2})$ time, where $n = \lceil \log_2 N \rceil$ is the input size in bits. The general number field sieve, the fastest known classical algorithm, runs in time $\exp\big(O(n^{1/3}(\log n)^{2/3})\big)$, which is subexponential, but still superpolynomial [Sho05]. No polynomial-time classical algorithm is known,

whether deterministic or randomized. Note, however, that factoring is in NP (a factor serves as a witness) and is not known to be NP-complete. In fact, it is widely conjectured not to be.

The quantum approach. Shor's algorithm [Sho97] solves the factoring problem in polynomial time on a quantum computer, running in $O(n^2 \log n)$ gate operations. We will focus on the two main ideas: the classical reduction from factoring to period-finding, and the quantum algorithm for period-finding via the quantum Fourier transform.

4.2.1 The Quantum Fourier Transform

The key quantum subroutine underlying Shor's algorithm is the *quantum Fourier transform* (QFT), the quantum analogue of the classical discrete Fourier transform.

Definition 23. Let $N = 2^n$ and let $\omega = e^{2\pi i/N}$. The **quantum Fourier transform** is the unitary operator on n qubits defined by

$$\mathrm{QFT}\,|x\rangle = \frac{1}{\sqrt{N}} \sum_{y=0}^{N-1} \omega^{xy}\,|y\rangle\,,$$

where x and y are treated as integers in the exponent. In matrix form:

$$\mathrm{QFT} = \frac{1}{\sqrt{N}} \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega & \omega^2 & \cdots & \omega^{N-1} \\ 1 & \omega^2 & \omega^4 & \cdots & \omega^{2(N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{N-1} & \omega^{2(N-1)} & \cdots & \omega^{(N-1)^2} \end{bmatrix}.$$

Each row k cycles through the Nth roots of unity k times around the unit circle, and the rows are mutually orthogonal, so the matrix is unitary.

The classical discrete Fourier transform on N elements takes $O(N \log N)$ operations via the fast Fourier transform (FFT). Since $N = 2^n$, this is $O(n \cdot 2^n)$, which is exponential in the input size. However, the quantum Fourier transform can be implemented by a quantum circuit of only $O(n^2)$ gates, an *exponential* speedup over the classical FFT. We omit the circuit construction (see [NC10], Section 5.1), but the key idea is that the QFT factors into a product of controlled phase rotations and Hadamard gates, exploiting the tensor product structure of the quantum state space.

The power of the QFT lies in its ability to efficiently extract periodicity from superpositions. If a quantum state has amplitudes that are periodic with some period r, then applying the QFT concentrates the amplitudes on multiples of N/r, allowing us to determine r by measurement. This is the mechanism at the heart of Shor's algorithm.

4.2.2 Reducing Factoring to Period-Finding

The first component of Shor's algorithm is entirely classical: a reduction from factoring to the problem of finding the *order* of an element modulo N.

Definition 24. The **order** of a modulo N (where $\gcd(a, N) = 1$) is the smallest positive integer r such that $a^r \equiv 1 \pmod{N}$.

The reduction proceeds as follows. Given N to factor:

1. Choose a random $a \in \{2, 3, \ldots, N-1\}$. If $\gcd(a, N) > 1$, we have found a factor and are done.
2. Otherwise, $\gcd(a, N) = 1$, so a has a well-defined order r modulo N. Find r (this is the hard step, which we will solve with quantum-specific methods).
3. If r is odd, return to step 1 and choose a new a (it can be shown that r is even with probability at least $\frac{1}{2}$ over the random choice of a).

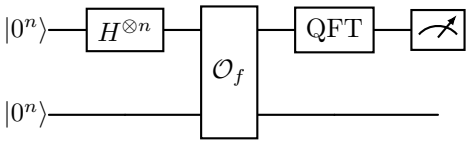
4. Compute $b = a^{r/2} \pmod{N}$. Since $b^2 = a^r \equiv 1 \pmod{N}$, we have $(b-1)(b+1) \equiv 0 \pmod{N}$. If $b \equiv -1 \pmod{N}$, return to step 1. Otherwise, at least one of $\gcd(b-1, N)$ and $\gcd(b+1, N)$ is a nontrivial factor of N.

Step 4 works because $(b-1)(b+1) \equiv 0 \pmod{N}$ means $N \mid (b-1)(b+1)$, so the prime factors of N must be distributed between $b-1$ and $b+1$. They cannot all divide the same one (since $b \not\equiv \pm 1 \pmod{N}$), so both gcds are nontrivial. It can be shown that the probability of needing to restart (due to r being odd or $b \equiv -1$) is at most $\frac{1}{2}$ per attempt, so the expected number of attempts is $O(1)$.

In particular, we can observe that factoring reduces to order-finding. And order-finding is a period-finding problem: the function $f(x) = a^x \bmod N$ is periodic with period r (the order of a), since $f(x+r) = a^{x+r} = a^x \cdot a^r \equiv a^x \equiv f(x) \pmod{N}$.

4.2.3 The Quantum Period-Finding Algorithm

We now describe how a quantum computer finds the period r of $f(x) = a^x \bmod N$ efficiently. The algorithm uses two registers: an n-qubit "input" register and an n-qubit "output" register, where $n = \lceil \log_2 N \rceil$.



The algorithm proceeds as follows:

Step 1: Create a uniform superposition. Apply $H^{\otimes n}$ to the first register:

$$|0^n\rangle\,|0^n\rangle \xrightarrow{H^{\otimes n} \otimes I} \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle\,|0^n\rangle\,.$$

Step 2: Compute f in superposition. Apply the oracle $\mathcal{O}_f : |x\rangle\,|0^n\rangle \mapsto |x\rangle\,|f(x)\rangle$:

$$\frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle\,|f(x)\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle\,|a^x \bmod N\rangle\,.$$

The oracle can be implemented efficiently: computing $a^x \bmod N$ by repeated squaring uses $O(n)$ modular multiplications, each costing $O(n^2)$ classical gate operations, which can be performed reversibly using the classical embedding from Section 3.4. The total cost is $O(n^3)$ gates.

Step 3: Apply the QFT. Apply the quantum Fourier transform to the first register. To see why this extracts the period, consider the state after Step 2. Since f has period r, the values $f(0), f(1), \ldots, f(r-1)$ are distinct and then repeat. For a fixed value $f(x_0)$ (where $0 \le x_0 < r$), the inputs mapping to this value are $x_0, x_0 + r, x_0 + 2r, \ldots$, which are a set of roughly N/r evenly spaced points. The QFT converts this periodic structure in the amplitudes into peaks at multiples of N/r in the frequency domain. Specifically, after the QFT, the first register has significant amplitude only on values y close to integer multiples of N/r.

Step 4: Measure and extract r. Measuring the first register yields a value $y \approx jN/r$ for some integer j. From $y/N \approx j/r$, we can recover r using the continued fractions algorithm: compute the continued fraction expansion of y/N and find the closest fraction with denominator at most N. With constant probability, this yields r (or a factor of r, in which case we can verify and retry). The continued fractions step is classical and runs in $O(n^2)$ time.

The total gate complexity is dominated by the modular exponentiation oracle at $O(n^3)$ gates, though more careful implementations achieve $O(n^2 \log n)$ [Sho97]. In either case, the algorithm runs in polynomial time, which means that:

Theorem 25 (Shor's algorithm [Sho97]). The factoring problem can be solved with high probability in $O(n^2 \log n)$ time on a quantum computer, where n is the number of bits in the input. In particular, FACTORING $\in$ BQP.

Remark 26 (Nontriviality of BQP). While factoring is not known to be NP-complete (and is widely conjectured not to be), the significance of Shor's algorithm lies in the *separation* it suggests between BQP and BPP. The best known classical algorithms for factoring are subexponential but superpolynomial, and decades of effort in computational number theory have not produced a polynomial-time classical algorithm. If factoring is indeed not in BPP (as is widely believed), then Shor's algorithm provides a concrete example of a problem in BQP $\setminus$ BPP, establishing that quantum computation is strictly more powerful than randomized classical computation. Even if factoring were eventually shown to be in BPP (which would be a major surprise), Shor's algorithm demonstrates that quantum algorithms can achieve exponential speedups over the *best currently known* classical algorithms, providing strong practical motivation for quantum computing.

Remark 27 (Consequences for cryptography). The security of widely deployed public-key cryptosystems rests on the presumed classical hardness of problems closely related to factoring. The RSA cryptosystem is directly based on the difficulty of factoring large integers: its security relies on the assumption that, given $N = pq$ for large primes p and q, no efficient algorithm can recover p and q. Elliptic curve cryptography (ECC), while not based on factoring directly, relies on the hardness of the discrete logarithm problem over elliptic curve groups, which Shor's algorithm also solves in polynomial time via a similar period-finding reduction. Thus, a sufficiently large fault-tolerant quantum computer would break both RSA and ECC. Current quantum hardware is far from this threshold, where factoring cryptographically relevant numbers (2048-bit RSA keys) would require on the order of millions of logical qubits with error correction, but the theoretical threat has motivated active research into **post-quantum cryptography**: cryptosystems whose security is based on problems believed to be hard even for quantum computers, such as lattice-based and code-based problems [Nat17].

4.3 BQP in the Classical Landscape

Even with our previous analysis of quantum algorithms for unstructured search and factoring, we don't get any definitive answers about how BQP relates to various classical complexity classes; at best, we have enough to form conjectures about the relative power of BQP compared to classes like BPP and NP. So far, the only inclusion we have established is the lower bound BPP $\subseteq$ BQP (Proposition 13), with no upper bound for BQP yet.

In this final subsection focused on the BQP class, we establish two upper bounds. The first, which we prove in full, is that BQP $\subseteq$ PSPACE. The second, whose proof we omit, is the stronger containment BQP $\subseteq$ PP. Together with the known classical inclusions, these give us the chain

$$\mathsf{BPP} \subseteq \mathsf{BQP} \subseteq \mathsf{PP} \subseteq \mathsf{PSPACE}.$$

One major open problem is whether BQP $\subseteq$ PH. It turns out that there is evidence *against* this containment, including oracle separations and problems in BQP that are not known to lie in PH.

Theorem 28 (Bernstein [BV93]). BQP $\subseteq$ PSPACE.

Proof. Let $L \in \mathsf{BQP}$, and let $\{Q_n\}_{n\in\mathbb{N}}$ be a uniform polynomial-size quantum circuit family deciding L. We describe a PSPACE algorithm that, on input x of length n, computes the acceptance probability of $Q_n(x)$ to sufficient precision.

The circuit Q_n acts on $m = \mathrm{poly}(n)$ qubits and consists of $T = \mathrm{poly}(n)$ gates $G_1, G_2, \ldots, G_T$. The final state is

$$|\psi\rangle = G_T G_{T-1} \cdots G_1 \left|x, 0^{m-n}\right\rangle,$$

and the acceptance probability is the sum of $|\alpha_z|^2$ over all basis states z where the output qubit is 1. It suffices to compute $\alpha_z = \langle z| G_T \cdots G_1 |x, 0^{m-n}\rangle$ for each basis state $z \in \{0,1\}^m$.

We compute each amplitude α_z by summing over all computational paths. Inserting a resolution of the identity $I = \sum_w |w\rangle \langle w|$ between every pair of consecutive gates:

$$\alpha_z = \sum_{w_1, w_2, \ldots, w_{T-1} \in \{0,1\}^m} \langle z| G_T |w_{T-1}\rangle \cdot \langle w_{T-1}| G_{T-1} |w_{T-2}\rangle \cdots \langle w_1| G_1 \left|x, 0^{m-n}\right\rangle .$$

This is a sum over $(2^m)^{T-1}$ paths (exponentially many), but we can evaluate it in polynomial space. The key observation is that each matrix entry $\langle w_i| G_i |w_{i-1}\rangle$ is efficiently computable: since G_i acts on at most 2 or 3 qubits (it is a gate from a fixed universal gate set), the entry is nonzero only if w_i and w_{i-1} agree on all qubits not acted on by G_i, and the nonzero entries are entries of a fixed constant-size unitary matrix with algebraic entries. Each such entry can be represented and manipulated in $O(\text{poly}(n))$ bits of precision.

The PSPACE algorithm iterates over all paths $w_1, \ldots, w_{T-1}$ one at a time (using $O(mT) = O(\text{poly}(n))$ space to store the current path), computes the product of matrix entries for each path (using $O(\text{poly}(n))$ space for the arithmetic), and accumulates the running sum for α_z. After processing all paths, it computes $|\alpha_z|^2$ and sums over all z with output qubit 1. The total space used is $O(\text{poly}(n))$: we need polynomial space for the current path, for the running product along the path, and for the running sum of amplitudes. Although the algorithm iterates over exponentially many paths, it reuses space between iterations. □

Theorem 29 (Adleman [ADH97])**.** BQP ⊆ PP.

Proof intuition. Recall that PP is the class of languages decidable by a polynomial-time nondeterministic Turing machine that accepts if and only if *more than half* of its computation paths accept. The key insight is that the sum-over-paths expression for a quantum amplitude can be reinterpreted as a counting problem. Specifically, the acceptance probability of a quantum circuit is a sum of exponentially many complex-valued terms (one per computational path), and the question of whether this sum exceeds $\frac{1}{2}$ can be encoded as the question of whether a majority of computation paths (in a suitably defined nondeterministic machine) are accepting. The main technical challenge is handling the complex phases: the path contributions are complex numbers, not just 0's and 1's, so the encoding requires a careful reduction that maps the complex arithmetic into a counting problem over $\{0,1\}$-valued paths.

For a full construction, refer to [ADH97]. Note, however, that an alternative proof follows elegantly from the result PostBQP = PP due to Aaronson [Aar04], which we mention in Section 6. □

5 QMA, QCMA, and the Local Hamiltonian Problem

Just as BQP is the quantum analogue of BPP, we can define quantum analogues of verification-based classes. In this section, we introduce QMA (the quantum analogue of MA/NP) and QCMA (a variant where the proof remains classical), and state the key completeness result for QMA: the local Hamiltonian problem.

The known inclusions among these classes are:

$$\text{NP} \subseteq \text{MA} \subseteq \text{QCMA} \subseteq \text{QMA} \subseteq \text{PP} \subseteq \text{PSPACE}.$$

The first two inclusions are standard from classical complexity theory. The inclusion MA ⊆ QCMA holds because a quantum verifier is at least as powerful as a classical probabilistic verifier. The inclusion QCMA ⊆ QMA holds because a classical proof string is a special case of a quantum state (a computational basis state). The inclusion QMA ⊆ PP is a nontrivial result due to Marriott and Watrous [MW05]. Whether any of these inclusions are strict is open; in particular, whether QMA = QCMA (i.e. whether quantum witnesses are genuinely more useful than classical ones) is a major open question.

5.1 Definition of QMA

Recall the class MA from classical complexity theory. In an MA protocol, an all-powerful prover (Merlin) sends a classical proof string to a probabilistic polynomial-time verifier (Arthur), who then decides whether to accept. Formally, a language $L \in \mathsf{MA}$ if there exists a polynomial-time probabilistic Turing machine V and a polynomial q such that:

- (Completeness) For all $x \in L$, there exists a string $z \in \{0,1\}^{q(|x|)}$ such that $\Pr[V(x,z)=1] \geq \frac{2}{3}$.
- (Soundness) For all $x \notin L$ and all strings $z \in \{0,1\}^{q(|x|)}$, $\Pr[V(x,z)=1] \leq \frac{1}{3}$.

QMA modifies this setup in two ways: the proof is now a *quantum state*, and the verifier is a *polynomial-time quantum computer*.

Definition 30. A language $L \in \mathsf{QMA}$ if there exists a polynomial-time quantum verifier V and a polynomial p such that:

- (Completeness) For all $x \in L$, there exists a quantum state $|\psi\rangle$ on $p(|x|)$ qubits such that $\Pr[V(|x\rangle, |\psi\rangle)=1] \geq \frac{2}{3}$.
- (Soundness) For all $x \notin L$ and all quantum states $|\psi\rangle$ on $p(|x|)$ qubits, $\Pr[V(|x\rangle, |\psi\rangle)=1] \leq \frac{1}{3}$.

As in the BPP and BQP cases, the constant $\frac{2}{3}$ is arbitrary and can be amplified. However, amplification for QMA is more subtle than for BQP: the obvious strategy of running the verifier multiple times on independent copies of the witness fails, because the no-cloning theorem (Theorem 6) prevents us from duplicating the quantum witness $|\psi\rangle$. One might ask Merlin to send multiple copies, but a dishonest Merlin could send an entangled state rather than independent copies. It turns out that amplification is still possible. In fact, Kitaev and Watrous [KW00] showed that the completeness and soundness parameters can be amplified to $1-2^{-\mathrm{poly}(n)}$ and $2^{-\mathrm{poly}(n)}$ respectively by using a more careful analysis of the verifier's acceptance operator.

5.2 Definition of QCMA

A natural intermediate class restricts the witness to be classical while keeping the verifier quantum.

Definition 31. A language $L \in \mathsf{QCMA}$ if there exists a polynomial-time quantum verifier V and a polynomial p such that:

- (Completeness) For all $x \in L$, there exists a classical string $y \in \{0,1\}^{p(|x|)}$ such that $\Pr[V(|x\rangle, |y\rangle)=1] \geq \frac{2}{3}$.
- (Soundness) For all $x \notin L$ and all classical strings $y \in \{0,1\}^{p(|x|)}$, $\Pr[V(|x\rangle, |y\rangle)=1] \leq \frac{1}{3}$.

The difference from QMA is that Merlin is restricted to sending a classical string rather than a quantum state. The verifier is still a quantum computer, so QCMA sits between MA (classical verifier, classical proof) and QMA (quantum verifier, quantum proof). Whether $\mathsf{QCMA} = \mathsf{QMA}$, that is, whether quantum proofs are strictly more powerful than classical proofs when verified by a quantum computer, remains one of the central open questions in quantum complexity theory.

5.3 The Local Hamiltonian Problem

Just as NP has a natural complete problem with SAT (Boolean satisfiability), QMA has a natural complete problem: the *local Hamiltonian problem*. First, recall the Hermitian operator (for which we have the following more detailed definition):

Definition 32. A **Hermitian operator** on a complex vector space V is a linear operator H satisfying $H^\dagger = H$ (equivalently, $\langle Hx, y\rangle = \langle x, Hy\rangle$ for all $x, y \in V$). Every Hermitian operator has real eigenvalues. The smallest eigenvalue of H is called the **ground state energy**, and a corresponding eigenvector is called the **ground state**.

Definition 33. An operator H on n qubits is a **k-local Hamiltonian** if it can be written as $H = \sum_{j=1}^{r} H_j$, where each H_j is a Hermitian operator acting nontrivially on at most k of the n qubits (and as the identity on the rest).

Definition 34. The promise problem k-LOCAL HAMILTONIAN is defined as follows. The input is a tuple (H, a, b) where:

- $H = \sum_{j=1}^{r} H_j$ is a k-local Hamiltonian on n qubits, with $r = \text{poly}(n)$ terms, each H_j satisfying $\|H_j\| \leq \text{poly}(n)$ and having entries specified with $\text{poly}(n)$ bits of precision,
- $a, b \in \mathbb{R}$ satisfy $b - a \geq 1/\text{poly}(n)$.

The promise is that the smallest eigenvalue of H is either at most a or at least b. The output is YES if the smallest eigenvalue is at most a, and NO if it is at least b.

The local Hamiltonian problem asks whether a structured Hermitian matrix has a small eigenvalue. The "locality" condition (k-local) is the quantum analogue of a constraint being over a bounded number of variables: just as a clause in a k-CNF formula involves at most k Boolean variables, a k-local term H_j involves at most k qubits. The ground state energy plays the role of the optimum value: a low ground state energy means the constraints can be approximately satisfied simultaneously, while a high ground state energy means they cannot.

Theorem 35 (Kitaev [KSV02]). The k-local Hamiltonian problem is QMA-complete for $k \geq 5$.

This result was subsequently strengthened: Kempe and Regev [KR03] proved QMA-completeness for $k \geq 3$, and Kempe, Kitaev, and Regev [KKR05] proved it for $k = 2$.

Containment in QMA. Showing that k-LOCAL HAMILTONIAN $\in$ QMA is the easier direction. The quantum witness is the ground state $|\psi\rangle$ itself. Given $|\psi\rangle$, the verifier estimates the energy $\langle\psi| H |\psi\rangle = \sum_j \langle\psi| H_j |\psi\rangle$ by randomly selecting a term H_j and measuring the energy contribution of that term (which involves only k qubits and can be done efficiently). Repeating and averaging gives a sufficiently precise estimate to distinguish the YES and NO cases.

QMA-hardness. The harder direction, showing that every QMA problem reduces to k-LOCAL HAMILTONIAN, is the quantum analogue of the Cook-Levin theorem. The proof, due to Kitaev [KSV02], uses a construction called the **circuit-to-Hamiltonian** mapping. The main idea is as follows. Given a QMA verifier V that acts via a sequence of gates $G_1, \ldots, G_T$ on the input and witness, Kitaev constructs a Hamiltonian whose ground state encodes the *entire history* of the verification computation. This is done using a "clock register" of T qubits that tracks the time step:

$$|\psi_{\text{history}}\rangle = \frac{1}{\sqrt{T+1}} \sum_{t=0}^{T} G_t \cdots G_1 |\psi_{\text{init}}\rangle \otimes |t\rangle_{\text{clock}},$$

where $|t\rangle_{\text{clock}}$ indicates the time step in unary encoding. The Hamiltonian H is then a sum of local terms that penalize:

- *Invalid transitions*: the computation register at time $t+1$ should equal G_{t+1} applied to the state at time t. Each such check is local because each gate acts on a constant number of qubits.
- *Incorrect initialization*: the input qubits should contain $|x\rangle$ and the ancillas should be $|0\rangle$ at time $t = 0$.
- *Rejection*: the output qubit should indicate acceptance at the final time step $t = T$.

If the verifier accepts some witness with high probability, then there exists a history state with low energy (the penalty terms are nearly satisfied). If the verifier rejects all witnesses, then every state incurs significant energy from at least one penalty term. The locality of the resulting Hamiltonian comes from the fact that each gate acts on a constant number of qubits and the clock register checks are local. The full proof, including the analysis of the promise gap, can be found at [KSV02].

The analogy to the Cook-Levin theorem is worth making explicit: just as Cook-Levin encodes the history of a nondeterministic Turing machine computation into a Boolean formula (with local clauses checking that each step follows from the previous one), Kitaev's construction encodes the history of a quantum verification into a local Hamiltonian (with local terms checking that each gate is applied correctly). The role of the satisfying assignment is played by the ground state, and the role of the formula's value (satisfied or not) is played by the ground state energy (low or high).

6 Conclusion and Open Problems

6.1 Key Takeaways

We have established the main results of quantum computing, defining qubits, quantum gates, and how they interact in the circuit model. With the equivalence to quantum turing machines, we have a well-defined method to establish BQP, QCMA, and QMA (using the circuit model) as complexity classes capturing the behavior of the hardness of quantum problems. Certain algorithms and problems display the nontriviality of quantum computing, establishing the power of these quantum complexity classes to the classical landscape. This includes Grover's algorithm (for unstructured search), Shor's algorithm (for factoring/period-finding), and the local Hamiltonian problem (for simulation of quantum systems).

Most of all, we observe that many of the paradigms and results we established in classical complexity theory had parallels to quantum complexity theory. This shows us that, even though the underlying physical mode of computation is very different, we can still apply similar techniques and methods in proving relationships between these complexity classes.

6.2 Recent Results

We will give brief introductions to other quantum complexity classes and some interesting results which have been proved recently about them.

6.2.1 PostBQP = PP

First, we consider PostBQP, the class of problems solvable in polynomial time on a quantum Turing machine with postselection. Postselection is the process of conditioning on the occurrence of a given event, i.e. changing the probability of an event E, $\Pr[E]$ to the conditional probability $\Pr[E|F]$, the probability of E occurring given another event F.

If we added postselection to BPP, we get a class called $\mathsf{BPP}_{\mathrm{PATH}}$ which is explored in [HHT97] and proved to be somewhere between MA and PP. On the other hand, when allowing for quantum Turing machines, Aaronson proves in [Aar04] that in fact PostBQP = PP, indicating that postselection is quite a strong condition, turning BQP, a class which is not known to contain NP, into PP which is thought to be much stronger than NP.

6.2.2 QIP = PSPACE

Another classical complexity class which can be made quantum is the class IP, the set of problems solvable by an interactive proof. In class, it was mentioned that IP = PSPACE (proved in [Sha92]). Now, allowing for quantum prover and verifier, the question is whether $\mathsf{QIP} \stackrel{?}{=} \mathsf{IP}$. This was proven be the case in 2009, when [Jai+09] proved that QIP $\subseteq$ PSPACE and thus QIP $\subseteq$ IP (i.e. QIP = IP). So, this is an instance where making our prover and verifier quantum does not actually make the system any more powerful, unlike many of the previous examples.

6.2.3 Evidence that BQP is not contained in PH

The relation between BQP and PH is currently unknown, but it is conjectured that BQP contains problems not in PH, i.e., BQP $\not\subset$ PH. There are two strong contenders, Fourier-Fishing and Fourier-Checking, contained in BQP but which have evidence for why they may not be contained in PH. As the descriptions for these problems are complicated and not important to this paper, we refer a curious reader to the article [Aar09].

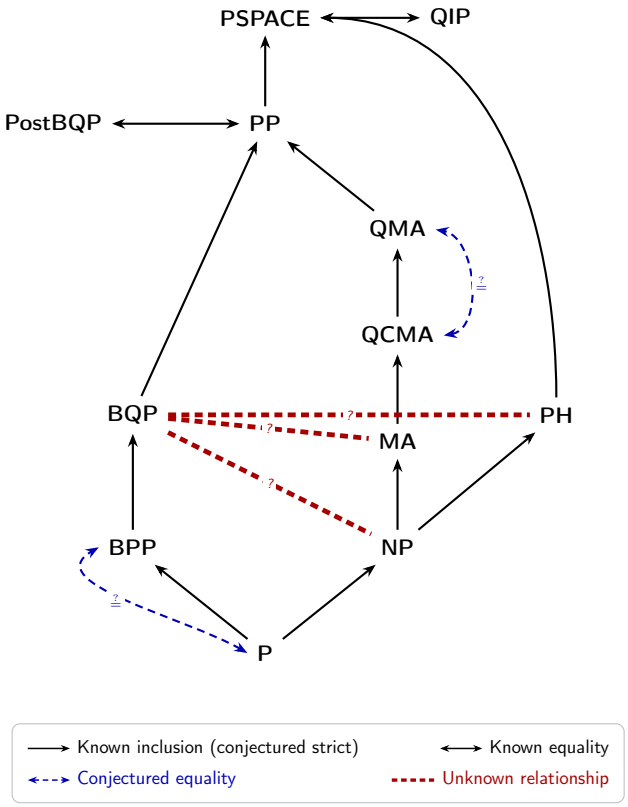
Further evidence for why we suspect BQP $\not\subset$ PH is described in the following concrete result:

Theorem 36 (Oracle Separation of BQP and PH)**.** There exists an oracle A such that $\mathsf{BQP}^A \not\subset \mathsf{PH}^A$.

Again, we will not prove this result, but one can read about it in the original paper [RT22].

6.3 What We Know + Open Problems

Throughout this paper, we covered many different quantum complexity classes, and also introduced many interesting and conjectured relationships between the quantum complexity classes and they classical complexity classes we have studied deeply. The diagram below summarizes these relationships, encoding many of the equalities and inclusions we know, as well as some major open problems that remain in the field of quantum complexity theory:



We hope that you enjoyed this journey through quantum complexity theory, all the way from the fundamental mathematics and quantum physics to the creative and surprising quantum algorithms and quantum complexity classes!

References

[Aar04] Scott Aaronson. *Quantum Computing, Postselection, and Probabilistic Polynomial-Time*. 2004. arXiv: quant-ph/0412187 [quant-ph]. URL: https://arxiv.org/abs/quant-ph/0412187.

[Aar09] Scott Aaronson. *BQP and the Polynomial Hierarchy*. 2009. arXiv: 0910.4698 [quant-ph]. URL: https://arxiv.org/abs/0910.4698.

[ADH97] Leonard M. Adleman, Jonathan DeMarrais, and Ming-Deh A. Huang. "Quantum Computability". In: *SIAM Journal on Computing* 26.5 (1997), pp. 1524–1540. DOI: 10.1137/S0097539795293639. eprint: https://doi.org/10.1137/S0097539795293639. URL: https://doi.org/10.1137/S0097539795293639.

[Ben73] C. H. Bennett. "Logical Reversibility of Computation". In: *IBM Journal of Research and Development* 17.6 (1973), pp. 525–532. DOI: 10.1147/rd.176.0525.

[Ben+97] Charles H. Bennett et al. "Strengths and Weaknesses of Quantum Computing". In: *SIAM Journal on Computing* 26.5 (1997), pp. 1510–1523. DOI: 10.1137/S0097539796300933. eprint: https://doi.org/10.1137/S0097539796300933. URL: https://doi.org/10.1137/S0097539796300933.

[BV93] Ethan Bernstein and Umesh Vazirani. "Quantum complexity theory". In: *Proceedings of the Twenty-Fifth Annual ACM Symposium on Theory of Computing*. STOC '93. San Diego, California, USA: Association for Computing Machinery, 1993, pp. 11–20. ISBN: 0897915917. DOI: 10.1145/167088.167097. URL: https://doi.org/10.1145/167088.167097.

[Boy+99] P. Oscar Boykin et al. *On Universal and Fault-Tolerant Quantum Computing*. 1999. arXiv: quant-ph/9906054 [quant-ph]. URL: https://arxiv.org/abs/quant-ph/9906054.

[Chi93] A. Chi-Chih Yao. "Quantum circuit complexity". In: *Proceedings of 1993 IEEE 34th Annual Foundations of Computer Science*. 1993, pp. 352–361. DOI: 10.1109/SFCS.1993.366852.

[DN05] Christopher M. Dawson and Michael A. Nielsen. *The Solovay-Kitaev algorithm*. 2005. arXiv: quant-ph/0505030 [quant-ph]. URL: https://arxiv.org/abs/quant-ph/0505030.

[Deu85] David Deutsch. "Quantum theory, the Church–Turing principle and the universal quantum computer". In: *Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences* 400.1818 (July 1985), pp. 97–117. ISSN: 0080-4630. DOI: 10.1098/rspa.1985.0070. eprint: https://royalsocietypublishing.org/rspa/article-pdf/400/1818/97/65933/rspa.1985.0070.pdf. URL: https://doi.org/10.1098/rspa.1985.0070.

[Fey82] Richard P. Feynman. "Simulating Physics with Computers". In: *International Journal of Theoretical Physics* 21.6–7 (1982), pp. 467–488. DOI: 10.1007/BF02650179.

[Gro96] Lov K. Grover. "A fast quantum mechanical algorithm for database search". In: *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*. STOC '96. Philadelphia, Pennsylvania, USA: Association for Computing Machinery, 1996, pp. 212–219. ISBN: 0897917855. DOI: 10.1145/237814.237866. URL: https://doi.org/10.1145/237814.237866.

[HHT97] Yenjo Han, Lane A. Hemaspaandra, and Thomas Thierauf. "Threshold Computation and Cryptographic Security". In: *SIAM Journal on Computing* 26.1 (1997), pp. 59–78. DOI: 10.1137/S0097539792240467. eprint: https://doi.org/10.1137/S0097539792240467. URL: https://doi.org/10.1137/S0097539792240467.

[Jai+09] Rahul Jain et al. *QIP = PSPACE*. 2009. arXiv: 0907.4737 [quant-ph]. URL: https://arxiv.org/abs/0907.4737.

[KKR05] Julia Kempe, Alexei Kitaev, and Oded Regev. *The Complexity of the Local Hamiltonian Problem*. 2005. arXiv: quant-ph/0406180 [quant-ph]. URL: https://arxiv.org/abs/quant-ph/0406180.

[KR03] Julia Kempe and Oded Regev. *3-Local Hamiltonian is QMA-complete*. 2003. arXiv: quant-ph/0302079 [quant-ph]. URL: https://arxiv.org/abs/quant-ph/0302079.

[KW00] Alexei Kitaev and John Watrous. "Parallelization, amplification, and exponential time simulation of quantum interactive proof systems". In: *Proceedings of the Thirty-Second Annual ACM Symposium on Theory of Computing*. STOC '00. Portland, Oregon, USA: Association for Computing Machinery, 2000, pp. 608–617. ISBN: 1581131844. DOI: 10.1145/335305.335387. URL: https://doi.org/10.1145/335305.335387.

[KSV02] Alexei Yu. Kitaev, Alexander H. Shen, and Mikhail N. Vyalyi. *Classical and Quantum Computation*. Vol. 47. Graduate Studies in Mathematics. Providence, RI: American Mathematical Society, 2002. ISBN: 978-0-8218-3229-5. DOI: 10.1090/gsm/047.

[Man80] Yuri I. Manin. *Vychislimoe i nevychislimoe [Computable and Noncomputable]*. In Russian. Moscow: Soviet Radio, 1980.

[MW05] Chris Marriott and John Watrous. *Quantum Arthur-Merlin Games*. 2005. arXiv: cs/0506068 [cs.CC]. URL: https://arxiv.org/abs/cs/0506068.

[Nat17] National Institute of Standards and Technology. *Post-Quantum Cryptography Standardization*. Accessed: 2026-03-11. 2017. URL: https://csrc.nist.gov/pqc-standardization.

[NC10] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2010.

[RT22] Ran Raz and Avishay Tal. "Oracle Separation of BQP and PH". In: *J. ACM* 69.4 (Aug. 2022). ISSN: 0004-5411. DOI: 10.1145/3530258. URL: https://doi.org/10.1145/3530258.

[Sha92] Adi Shamir. "IP = PSPACE". In: *J. ACM* 39.4 (Oct. 1992), pp. 869–877. ISSN: 0004-5411. DOI: 10.1145/146585.146609. URL: https://doi.org/10.1145/146585.146609.

[Sho97] Peter W. Shor. "Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer". In: *SIAM Journal on Computing* 26.5 (Oct. 1997), pp. 1484–1509. ISSN: 1095-7111. DOI: 10.1137/s0097539795293172. URL: http://dx.doi.org/10.1137/S0097539795293172.

[Sho05] Victor Shoup. *A Computational Introduction to Number Theory and Algebra*. Cambridge University Press, 2005.

[WZ82] William K. Wootters and Wojciech H. Zurek. "A Single Quantum Cannot Be Cloned". In: *Nature* 299 (1982), pp. 802–803. DOI: 10.1038/299802a0.